LAB: Using NPM – End of module Assessment

Explorador de la foto astronómica del día de la NASA (APOD)

1. ¿Qué vas a construir?

La NASA publica cada día la **Astronomy Picture of the Day (APOD)**: una imagen o video del universo (galaxias, nebulosas, planetas, eclipses…) con una explicación científica escrita por astrónomos.

Vas a construir, **desde cero**, una página web que:

- Muestra la **foto del día** apenas se abre.
- Permite **buscar la APOD de cualquier fecha** desde el 16 de junio de 1995.
- Deja **guardar favoritas** que siguen ahí aunque recargues la página.

Y lo vas a hacer como se trabaja en un proyecto real: con **npm** para manejar las librerías del proyecto.

Lo que vas a practicar

Tema	Dónde lo usas
npm: `npm init`, `npm install`, `package.json`, `node_modules`	Preparar el proyecto e instalar la librería **Day.js**
fetch + **async/await** + **try/catch**	Pedirle las fotos a la API de la NASA
DOM y **eventos**	Mostrar la foto y reaccionar a botones y clics
localStorage + **JSON**	Guardar los favoritos
Validaciones	No permitir fechas vacías, futuras o anteriores a 1995

Antes de empezar

Necesitas **Node.js** instalado (trae npm incluido), **VS Code** y la extensión **Live Server**. Verifica Node y npm en la terminal de VS Code:

📄 **Terminal de VS Code**

```
node -v
npm -v
```

Si cada comando muestra un número de versión, estás listo. Si sale el error **“no se reconoce como un comando”**, Node.js no está instalado: descárgalo de nodejs.org (versión LTS), cierra VS Code y vuelve a abrirlo.

© 2026 Ana Alvarado · Educadora Tech & Desarrolladora Full Stack · Todos los derechos reservados · linkedin.com/in/ana-alvarado-instructora-full-stack

2. Paso 0 — Preparar el proyecto con npm

2.1 Crear la carpeta y el package.json

Crea una carpeta llamada `explorador-apod`, ábrela en VS Code (**Archivo → Abrir carpeta**) y abre la terminal (**Terminal → Nueva terminal**). Ejecuta:

📄 **Terminal (dentro de explorador-apod)**

```
npm init -y
```

Esto crea el archivo `package.json`: la **"hoja de vida" del proyecto**. Ahí queda el nombre, la versión y, más adelante, la lista de librerías que usa. La `-y` acepta todas las opciones por defecto.

2.2 Instalar Day.js

Day.js es una librería muy liviana para trabajar con fechas. La vas a necesitar porque la API de la NASA pide las fechas en un formato especial y además debes validar fechas futuras. Instálala:

📄 **Terminal**

```
npm install dayjs
```

Revisa qué cambió en tu proyecto:

- Apareció la carpeta `node_modules/`: ahí está el código de Day.js.
- En `package.json` apareció la sección `"dependencies"` con `"dayjs"`.
- Apareció `package-lock.json`, que guarda la versión exacta instalada.

2.3 Crear el .gitignore

`node_modules` puede llegar a pesar muchísimo y **no se sube a GitHub**. Quien descargue tu proyecto solo ejecuta `npm install` y npm vuelve a descargar todo leyendo el `package.json`.

Crea en la raíz del proyecto un archivo llamado exactamente `.gitignore` (con el punto al inicio) con este contenido:

📄 **.gitignore (nuevo, en la raíz de explorador-apod)**

```
node_modules/
```

2.4 Estructura del proyecto

Crea las carpetas y archivos que faltan para que tu proyecto quede así:

© 2026 Ana Alvarado · Educadora Tech & Desarrolladora Full Stack · Todos los derechos reservados · linkedin.com/in/ana-alvarado-instructora-full-stack

```
explorador-apod/
├── css/
│   └── styles.css
├── js/
│   └── app.js
├── node_modules/        ← lo creó npm, no lo toques
├── .gitignore
├── index.html
├── package.json
└── package-lock.json
```

2.5 Conectar Day.js en el HTML

En `index.html` crea la estructura básica y, **al final del** `<body>`, agrega tres scripts en este orden. El orden importa: primero se carga Day.js, luego su idioma español y por último tu código, que ya puede usarlos.

📄 **index.html (justo antes de cerrar </body>)**

```
 <script src="node_modules/dayjs/dayjs.min.js"></script>
  <script src="node_modules/dayjs/locale/es.js"></script>
  <script src="js/app.js"></script>
</body>
```

2.6 Comprobar que todo funciona

Escribe esto en `app.js`, abre `index.html` con **Live Server** (clic derecho → Open with Live Server) y mira la consola del navegador (F12 → Console):

📄 **js/app.js**

```
dayjs.locale("es");
console.log(dayjs().format("D [de] MMMM [de] YYYY"));
```

Debe aparecer la fecha de hoy en español, por ejemplo **29 de octubre de 2026**.

> **Si sale** `dayjs is not defined`**:** el navegador no encontró la librería. Revisa que la ruta del `<script>` esté bien escrita, que `index.html` esté en la raíz del proyecto (al lado de `node_modules`) y que el script de Day.js esté **antes** del de `app.js`.

© 2026 Ana Alvarado · Educadora Tech & Desarrolladora Full Stack · Todos los derechos reservados · linkedin.com/in/ana-alvarado-instructora-full-stack

3. La API que vas a usar

La NASA tiene una API pública de APOD. **No necesitas API key** ni registrarte.

¿Qué necesitas?	URL	¿Qué responde?
La foto de **hoy**	`https://science.nasa.gov/wp-json/wp/v2/apod-basic?per_page=1`	Un **arreglo** con una APOD adentro → usa `datos[0]`
La foto de **una fecha**	`https://science.nasa.gov/wp-json/wp/v2/apod-basic/AAMMDD` Ejemplo 15 de marzo de 2024 → `.../apod-basic/240315`	Un **objeto** con la APOD
Una fecha **sin foto**	Igual que la anterior	Error **404** (`respuesta.ok` es `false`)

Truco: pega cualquiera de esas URL en el navegador y verás el JSON que recibe tu código.

Los campos que vas a usar

Campo	Qué trae	Ejemplo
`title`	Título de la APOD (en inglés)	"The Horns of Venus"
`date`	Fecha en formato AAAA-MM-DD	"2026-10-29"
`hdurl`	**La imagen** → va en el `src` del `<img>`	"https://assets.science.nasa.gov/...jpg"
`media_type`	Tipo de publicación	"image" o "video"
`explanation`	Explicación científica **en HTML** (trae negritas y enlaces)	"Explanation: ..."
`permalink`	Enlace a la publicación en el sitio de la NASA	"https://science.nasa.gov/image-article/..."

> **Ojo con estas trampas:**
> • El campo `url` **no es la imagen**: es la página del artículo. Usa `hdurl`.
> • Como `explanation` viene con HTML, muéstrala con `innerHTML` (con `textContent` verías las etiquetas).
> • Si en internet encuentras tutoriales con `api.nasa.gov/planetary/apod` y `DEMO_KEY`, **no los sigas**: esa API vieja ya no funciona, devuelve el logo de la NASA en vez de la foto.

4. Funcionalidad 1 — La foto del día

Requisitos

- Al abrir la página se carga automáticamente la APOD de hoy.

© 2026 Ana Alvarado · Educadora Tech & Desarrolladora Full Stack · Todos los derechos reservados · linkedin.com/in/ana-alvarado-instructora-full-stack

- Se muestra en pantalla:
  - **Título**.
  - **Fecha en español** usando Day.js (ej. 29 de octubre de 2026).
  - **Imagen** (con `alt` igual al título).
  - **Explicación** científica.
- Mientras llega la respuesta se ve el mensaje **“Cargando…”**.
- Si algo falla, el error se muestra **en la página** (no solo en la consola).

Pistas

La petición va en una función `async` con `try/catch`. Recuerda que `fetch` **no** lanza error con un 404: tú debes revisar `respuesta.ok` y lanzarlo con `throw`.

📄 **js/app.js (dentro de tu función async)**

```
const respuesta = await fetch(url);
if (!respuesta.ok) {
  throw new Error("No hay APOD para esa fecha");
}
const datos = await respuesta.json();
```

Para la fecha en español, activa el idioma una sola vez al inicio de `app.js` y luego usa `format`:

📄 **js/app.js**

```
dayjs.locale("es");
dayjs(apod.date).format("D [de] MMMM [de] YYYY");
```

¿Y si es un video? Cuando `media_type` sea `"video"`, `hdurl` trae una imagen de vista previa. Muéstrala igual y agrega un enlace **“Ver el video en NASA”** que abra el `permalink` en otra pestaña. Cuando sea imagen, ese enlace no se debe ver.

5. Funcionalidad 2 — Buscar por fecha

Requisitos

- Un `<input type="date">` y un botón **Buscar**.
- Antes de llamar a la API, valida y muestra un mensaje en pantalla si:
  - No se eligió ninguna fecha.
  - La fecha es **futura** (la NASA aún no la tiene).
  - La fecha es **anterior al 16 de junio de 1995** (la primera APOD).
- Si la fecha es válida, se carga y se muestra esa APOD (reutiliza la función de la funcionalidad 1).

© 2026 Ana Alvarado · Educadora Tech & Desarrolladora Full Stack · Todos los derechos reservados · linkedin.com/in/ana-alvarado-instructora-full-stack

Pistas con Day.js

El input entrega la fecha como `"2024-03-15"`, pero la API la necesita como `240315`. Day.js lo convierte:

📄 **js/app.js**

```
dayjs("2024-03-15").format("YYMMDD");   // "240315"
```

Para validar, Day.js compara fechas por ti:

📄 **js/app.js**

```
dayjs(fecha).isAfter(dayjs(), "day")             // true si es futura
dayjs(fecha).isBefore(dayjs("1995-06-16"), "day") // true si es muy antigua
```

> **Plus de usabilidad:** el input acepta los atributos `min` y `max`. Pon `min="1995-06-16"` en el HTML y asigna el `max` desde JavaScript con `dayjs().format("YYYY-MM-DD")`, así el calendario no deja escoger días futuros. **Igual debes validar en JS**, porque la fecha se puede escribir a mano.

6. Funcionalidad 3 — Favoritos

Requisitos

- Un botón ⭐ **Guardar en favoritos** que guarda la APOD que se está viendo.
- Guarda solo lo necesario de cada favorito: `date` y `title` (no toda la explicación).
- Los favoritos se guardan en **localStorage** y siguen ahí al recargar la página.
- **No se permiten duplicados**: si la APOD ya está guardada, muestra un mensaje y no la agregues otra vez.
- Se muestra la lista de favoritos; **al hacer clic en uno se carga esa APOD**.

Pistas

- Guarda en una variable la APOD que se está mostrando (por ejemplo `apodActual`) para saber cuál guardar.
- localStorage solo guarda texto: usa `JSON.stringify` al guardar y `JSON.parse` al leer.
- Si todavía no hay nada guardado, `localStorage.getItem` devuelve `null`: en ese caso arranca con un arreglo vacío `[]`.
- Para revisar duplicados, el método `some()` te dice si algún favorito ya tiene la misma `date`.

© 2026 Ana Alvarado · Educadora Tech & Desarrolladora Full Stack · Todos los derechos reservados · linkedin.com/in/ana-alvarado-instructora-full-stack

7. ¿Cómo sé que funciona? — Pruebas

Antes de entregar, haz cada prueba y verifica el resultado esperado:

#	Haz esto	Debe pasar
1	Abre la página	Aparece la APOD de hoy con título, fecha en español, imagen y explicación
2	Busca el **4 de julio de 2015**	Aparece **“Aurora Australis”**
3	Busca el **13 de septiembre de 2026**	Es un video: se ve la imagen de vista previa y el enlace **“Ver el video en NASA”**
4	Presiona Buscar sin elegir fecha	Mensaje pidiendo elegir una fecha
5	Escribe una fecha futura	Mensaje de fecha futura; no se llama a la API
6	Busca el **1 de enero de 1990**	Mensaje de fecha anterior a 1995
7	Busca el **17 de junio de 1995**	Ese día no hubo APOD (404): mensaje **“No hay APOD para esa fecha”** y la página no se rompe
8	Guarda la misma APOD dos veces	Aparece una sola vez en la lista y sale un mensaje
9	Recarga la página (F5)	Los favoritos siguen en la lista
10	Haz clic en un favorito	Se carga esa APOD

> **Para que no te asustes:** las explicaciones vienen en inglés, y algunas fechas muy antiguas (sobre todo antes de 2001) muestran una imagen genérica de la NASA en vez de la foto original. Eso viene así desde la API, no es un error de tu código.

8. Entrega

- Sube el proyecto a un **repositorio de GitHub** y entrega el enlace por el canal que indique tu instructora.
- El repositorio debe tener:
  - `index.html`, `css/styles.css` y `js/app.js`.
  - `package.json` con `dayjs` en `"dependencies"`, y `package-lock.json`.
  - `.gitignore` con `node_modules/`, es decir, **sin** la carpeta `node_modules` en GitHub.

© 2026 Ana Alvarado · Educadora Tech & Desarrolladora Full Stack · Todos los derechos reservados · linkedin.com/in/ana-alvarado-instructora-full-stack

Checklist antes de entregar

✓	Criterio
☐	El proyecto se creó con `npm init` y Day.js se instaló con `npm install`
☐	`node_modules` no está en el repositorio
☐	La APOD de hoy carga al abrir la página, con su fecha en español
☐	La búsqueda por fecha funciona y valida fecha vacía, futura y anterior a 1995
☐	Los videos muestran la vista previa y el enlace al sitio de la NASA
☐	Los errores (como el 404) se muestran en pantalla y la página no se rompe
☐	Los favoritos se guardan, no se duplican, sobreviven a recargar y cargan la APOD al hacer clic
☐	El código está ordenado: funciones con nombres claros y cada una con una sola tarea
☐	Pasaste las 10 pruebas de la sección 7

Retos extra (opcionales)

- Botones ◀ **Día anterior** y **Día siguiente** ▶: con `dayjs(fecha).subtract(1, "day")` y `.add(1, "day")`. El botón de día siguiente no debe dejar pasar de hoy.
- Botón para **eliminar** un favorito de la lista (y de localStorage).
- Dale estilo con **Flexbox o Grid** para que se vea bien en celular.

Documentación oficial

- npm init: docs.npmjs.com/cli/commands/npm-init
- npm install: docs.npmjs.com/cli/commands/npm-install
- Day.js (formato): day.js.org/docs/en/display/format
- Day.js (comparar fechas): day.js.org/docs/en/query/is-after
- Fetch (MDN): developer.mozilla.org/es/docs/Web/API/Fetch_API/Using_Fetch
- localStorage (MDN): developer.mozilla.org/es/docs/Web/API/Window/localStorage
- input date (MDN): developer.mozilla.org/es/docs/Web/HTML/Element/input/date

© 2026 Ana Alvarado · Educadora Tech & Desarrolladora Full Stack · Todos los derechos reservados · linkedin.com/in/ana-alvarado-instructora-full-stack